Introduction to Recommender Systems

adm@cs.uni.wroc.pl

April 14, 2026

General remark:
Your solution should take the form of a short report in Jupyter Notebook, containing:

- codes and results for all elements of the solution,
- answers to the questions asked,
- and conclusions from each task.

When working with the data, you can find the docs very helpful.

Task 1 [2 points]

1. Download the data from the category Sports and Outdoors: https://cseweb.ucsd.edu/ jmcauley/datasets/amazon_v2/. Load the reviews into a `pd.DataFrame` with columns `['user_id', 'asin', 'rating', 'timestamp']`. Take care of the memory usage of your implementation.
2. What is the 5-core version?
3. Provide a basic data analysis for the full variant and the 5-core version:
   (a) number of interactions for each item (sort the counts for readability),
   (b) the distribution of users' number of interactions,
   (c) calculate the sparsity (how many of the possible user-item pairs are present in the data),
   (d) check for duplicates of contradictory data,
   (e) other plots that you find interesting.
4. Consider the following train-test split procedures. What are their advantages and disadvantages? Discuss it in terms of user and item representation, biases, leakage, and suitability for the ranking task.
   - Randomly select 20% of all interactions.
   - Randomly select 20% of users and put all their interactions in the test set.
   - Select 20% of interactions for each user at random.
   - Select the most recent 20% of interactions for each user.
   - Select a fixed number of the most recent interactions for each user to obtain around 20% of the data.

- Select the most recent 20% of all interactions.

5. Use the 5-core variant. Perform a train-test split by selecting the 2 most recent items for each user for testing.
6. Discuss how you can transform the data into boolean feedback.

You can obtain a point for the implementation and a point for the comments and answers.

Task 2 [3 points]

1. Implement a baseline model that recommends the most popular items.
2. Provide an implementation of the Matrix Factorization model (suggested using PyTorch).
3. Adapt the implementation of your RS model for using of the following loss functions:
   - MSE (as in Probabilistic MF):

$$\mathcal{L}_{MSE} = \sum_{(u,i)} \mathbb{1}_{ui} \left(\frac{r_{ui} - 1}{R_{max} - 1} - \sigma(\mathbf{p_u} \times \mathbf{q_i}) \right)^2 + \lambda_U \sum_u \|\mathbf{p_u}\|^2 + \lambda_I \sum_i \|\mathbf{q_i}\|^2$$

   - BPR:

$$\mathcal{L}_{BPR} = \sum_{(u,i,j)} \ln \sigma(\mathbf{p_u} \times \mathbf{q_i} - \mathbf{p_u} \times \mathbf{q_j}) - \lambda_\Theta \|\Theta\|^2$$

   - Alignment & Uniformity (remember to normalize the embeddings, see https://arxiv.org/abs/2206.12811 Eq 6):

$$\mathcal{L}_{AU} = \mathbb{E}_{(u,i)\sim\mathcal{D}_{positive}} \|\mathbf{p_u} - \mathbf{q_i}\|^2 + \lambda \left(\log \mathbb{E}_{(u,v)} \frac{1}{2} e^{-2\|\mathbf{p_u} - \mathbf{p_v}\|^2} + \log \mathbb{E}_{(i,j)} \frac{1}{2} e^{-2\|\mathbf{q_i} - \mathbf{q_j}\|^2} \right)$$

   Are there any data transformations needed to apply those loss functions? Is sampling negative examples necessary, and how can you perform it? Discuss possible options and implement one of them.
4. Select appropriate hyper-parameters to avoid underfitting and overfitting during training. If needed, use mini-batches for training. Monitor the training process (logs, wandb, ...).
5. For each loss function, use the embedding dimensionality of $16, 32, 64, 128$.
6. Compare the performance of the implemented models in terms of:
   - convergence speed,
   - recall@20,
   - hit-rate@20,
   - Normalized Discounted Cumulative Gain @20,
   - Mean Reciprocal Rank @20.

If you find the second task computationally too heavy, you can switch to Movielens-100k https://grouplens.org/datasets/movielens/100k/ or Movielens-1M.

You can obtain a point for implementing the popularity-based baseline with evaluation, a point for complete experiments with one of the mentioned loss functions, and half a point for experiments with each additional loss function.